REPORTE DE SEGURIDAD

BebidaHub

Análisis de controles, validación y endurecimiento de una aplicación full stack.

Proyecto individual · Desarrollo Web Integral
Autor: Héctor Hernández · GitHub @Hector9229
Fecha de generación: 06/11/2026
Backend: Django 5.2 · Django REST Framework 3.18 · SimpleJWT
Frontend: React 19 · Vite 6 · React Router 7

Documento generado automáticamente a partir del código, las capturas y la evidencia de ejecución del proyecto.

1. Resumen ejecutivo

BebidaHub es una aplicación web para administrar una colección personal de bebidas. El sistema expone una API REST protegida con JWT, aísla los datos por propietario y consume la API pública TheCocktailDB exclusivamente desde el backend. Este documento describe los controles de seguridad implementados, las verificaciones ejecutadas y los hallazgos corregidos durante el desarrollo.

El resultado de la revisión final es satisfactorio: **0 vulnerabilidades conocidas** en las dependencias auditadas (40 paquetes Python y 145 paquetes JavaScript), **39 pruebas automatizadas** en estado OK, cobertura backend de 88 % y **cero advertencias** en `check --deploy` con la configuración de producción.

Área	Estado final
Autenticación	JWT con access de 30 min, refresh de 1 día, rotación y blacklist
Autorización	IsAuthenticated + filtro por propietario en todas las consultas
Validación	Serializers estrictos, rangos, URL, HTML y coherencia de datos
Transporte	HTTPS, HSTS de 1 año, cookies Secure y redirección SSL en producción
Dependencias	pip-audit: 0 vulnerabilidades · npm audit: 0
Despliegue	check --deploy sin advertencias

2. Alcance y superficie de ataque

La superficie de ataque se compone de tres frentes: la SPA de React, la API de Django y la integración con un proveedor externo. El navegador solo puede alcanzar los endpoints publicados por la API; no tiene acceso directo a la base de datos ni a TheCocktailDB. La comunicación sensible viaja en el encabezado Authorization.

Componente	Exposición	Controles principales
SPA React	Pública	Rutas protegidas, sesión en cliente, sin secretos
API Django	Pública/autenticada	JWT, permisos, validación, throttling, CORS
Base de datos	Interna	Aislamiento por owner, constraints, PROTECT
TheCocktailDB	Salida del backend	Timeout, validación, caché, errores 502

Flujo de una petición autenticada

React envía el access token en `Authorization: Bearer ...`. SimpleJWT valida firma y expiración; DRF aplica permisos y throttling; el ViewSet filtra el queryset por `request.user`. Si el access token expiró, Axios intenta renovarlo una vez con el refresh token y reintenta la petición.

3. Controles implementados

3.1 Secretos y configuración por entorno

- La SECRET_KEY se lee de `backend/.env` y la aplicación falla al iniciar si no existe.
- Los archivos .env están excluidos por .gitignore; solo se versionan los ejemplos con placeholders.
- Desarrollo y producción usan módulos separados: `development.py` y `production.py`.
- Los valores sensibles de CORS, hosts, HSTS y proxy TLS también provienen de variables de entorno.

3.2 Autenticación y gestión de sesión

- Contraseñas con PBKDF2, mínimo 10 caracteres y validadores de similitud, contraseñas comunes y numéricas.
- Access token de 30 minutos y refresh token de 1 día.

- Rotación de refresh tokens y blacklist después de la rotación.
- Logout autenticado que invalida el refresh token en la base de datos.
- Mensaje genérico de credenciales incorrectas para no confirmar si un correo existe.
- Normalización de correo a minúsculas y comparación case-insensitive en el registro.

3.3 Autorización y aislamiento de datos

- Todas las operaciones de bebidas requieren autenticación.
- El queryset se filtra por propietario antes de cualquier operación de lectura o escritura.
- Modificar o eliminar una bebida ajena devuelve 404, no 403, para no revelar su existencia.
- El propietario se asigna en el servidor al crear; el cliente no puede enviarlo.
- Las categorías son globales de solo lectura y no se pueden borrar si tienen bebidas.

3.4 Validación y saneamiento de entrada

- Nombres de 2 a 100 caracteres, sin HTML ni caracteres de control, y únicos por usuario.
- Precio entre 0.00 y 999999.99; ABV entre 0 y 100; porción entre 30 y 2000 ml; rating entre 1 y 5.
- La descripción rechaza etiquetas HTML y se limita a 1000 caracteres.
- La URL de imagen solo acepta http/https y rechaza hosts locales, .local/.internal, loopback, privados y reservados.
- La búsqueda externa se limita a 2–50 caracteres y rechaza HTML y caracteres de control.
- El ordenamiento usa una allowlist; nunca se interpola el parámetro del usuario en SQL.
- Se fuerza coherencia entre ABV y la bandera de bebida alcohólica.

3.5 CORS, CSRF y cabeceras

- CORS con allowlist explícita de orígenes, sin credenciales y con métodos/cabeceras acotados.
- CSRF_TRUSTED_ORIGINS configurado para el frontend local y el dominio de producción.
- Cookies de sesión y CSRF HttpOnly, Secure y SameSite=Lax en producción.
- X-Content-Type-Options: nosniff, Referrer-Policy strict-origin-when-cross-origin, COOP same-origin y X-Frame-Options DENY.
- HSTS de un año con includeSubDomains y preload; SECURE_SSL_REDIRECT activo en producción.

3.6 Límites de abuso y disponibilidad

- Throttling anónimo de 80/min y autenticado de 240/min.
- Login y refresh limitados a 30/min; registro a 100/hora.
- API externa limitada a 40/min por usuario/IP.
- Cuerpos JSON/formulario limitados a 1 MiB y timeout de 8 s hacia el proveedor externo.
- Respuestas del proveedor externo se validan antes de normalizarse; los fallos se convierten en 502.

3.7 Dependencias y cadena de suministro

- Versiones fijadas en requirements.txt y package-lock.json.
- Auditorías automatizadas con pip-audit y npm audit.
- Actualización de Django REST Framework 3.16.1 a 3.18.3 para corregir avisos de seguridad.
- Actualización de React Router 6.30.1 a 7.18.5 para corregir avisos de seguridad.

4. Verificación y evidencia

Los siguientes resultados se obtuvieron con la configuración de producción y se conservan en `docs/evidence/`.

Verificación	Comando	Resultado
Deploy check	manage.py check --deploy --settings=config.settings.production	Sin advertencias
Pruebas backend	manage.py test	39 pruebas · OK · 29.562 s
Cobertura	coverage report	88 %
Dependencias Python	pip-audit --format json	40 deps · 0 vulnerabilidades
Dependencias JS	npm audit --json	145 deps · 0 vulnerabilidades
Build frontend	npm run build	Correcto

Salida del deploy check

```
System check identified no issues (0 silenced).
```

Fragmento de la ejecución de pruebas

```
Found 39 test(s). Ran 39 tests in 29.562s OK
```

Auditoría de dependencias

```
pip-audit: 40 dependencias analizadas, 0 vulnerabilidades conocidas. npm audit: 145 dependencias analizadas, 0 vulnerabilidades conocidas.
```

5. Hallazgos y remediación

Durante la auditoría inicial aparecieron avisos en dos dependencias. Ambos se corrigieron actualizando a versiones parchadas y se volvió a ejecutar la auditoría.

Hallazgo	Impacto potencial	Remediación	Estado
5 avisos en DRF 3.16.1: límites de cuerpo, negociación de Accept, AdminRenderer y revocación de tokens	Disponibilidad y divulgación limitada en rutas concretas	Actualización a DRF 3.18.3	Resuelto
2 avisos moderados en React Router 6.30.1: redirección abierta y deserialización en SSR	Redirección abierta; SSR no usado por esta SPA	Actualización a React Router 7.18.5	Resuelto

5.1 Riesgos residuales

Riesgo	Nivel	Mitigación actual	Recomendación
Tokens JWT en localStorage	Medio	Salida escapada por React, sin dangerouslySetInnerHTML, backend rechaza HTML	Añadir CSP estricta y valorar cookies HttpOnly
Credenciales demo públicas	Bajo	Cuenta limitada a datos de demostración	Deshabilitar en producción real
SQLite en demostración	Bajo	Suficiente para el alcance académico	Migrar a PostgreSQL en producción
Caché LocMem por proceso	Bajo	TTL de 5 minutos y límite de peticiones	Usar Redis si se escala horizontalmente
Dependencia de TheCocktailDB	Bajo	Timeout, validación, caché y error 502 controlado	Monitorizar disponibilidad y aplicar fallback
Sin MFA	Bajo	Contraseñas robustas y throttling de login	Añadir segundo factor para cuentas sensibles

6. Lista de verificación para despliegue

- DEBUG=False y DJANGO_SETTINGS_MODULE=config.settings.production.
- SECRET_KEY única, larga y fuera del repositorio.
- ALLOWED_HOSTS, CORS_ALLOWED_ORIGINS y CSRF_TRUSTED_ORIGINS con dominios reales.
- HTTPS terminado en proxy y cabecera X-Forwarded-Proto configurada.
- Base de datos PostgreSQL y copias de seguridad programadas.

- collectstatic y servicio de estáticos configurados.
- pip-audit y npm audit sin vulnerabilidades antes de publicar.
- check --deploy sin advertencias.
- CSP y límites de tasa del proxy como defensa en profundidad.

7. Conclusión

BebidaHub implementa un conjunto coherente de controles para una aplicación académica full stack: autenticación robusta, aislamiento de datos, validación estricta, configuración endurecida de producción, límites de abuso y auditoría de dependencias. Los avisos detectados fueron remediados y la verificación final no reporta vulnerabilidades conocidas ni advertencias de despliegue. Los riesgos residuales documentados son acotados y tienen una ruta clara de mejora.

Este reporte refleja el estado del proyecto en la fecha de generación. Las auditorías de dependencias deben repetirse periódicamente.